Ticket to Ride
Design Document

Authors:
Chay Hong Hong
Josh Brian Saputra
Simao Huang

Team: DevOops

Document Revision History

Date	Version	Description	Author
25/04/2026	0.1	Created initial Sprint 2 report document structure	Josh Brian Saputra
29/04/2026	0.2	Completed backend/game logic implementation details for the report	Josh Brian Saputra
04/05/2026	0.3	Completed frontend/UI implementation details for the report	Simao Huang
04/05/2026	0.4	Added team information object-oriented design explanation, and design rationales	Chay Hong Hong
04/05/2026	1.0	Finalised report writing, formatting, and consistency checks	Chay Hong Hong

1 Introduction

This document presents the software design for the *Ticket to Ride* system developed by Team DevOops. The purpose of this document is to provide a clear and structured overview of the system architecture, object-oriented design, and key design decisions made during Sprint 2.

The document is organised as follows:

- **Team Information** - details of team members, roles, and contributions
- **Object-Oriented Design** - UML class diagrams and system structure
- **Design Rationales** - explanation and justification of design decisions
- **Supplementary Documentation** - additional supporting materials

This document is intended to be sufficiently detailed so that another development teacher can understand and implement the system based solely on this design.

2 Team Information

2.1 Team Name and Membership

Team Name: DevOops

Team Member	Contact Details	Sprint 1 Role / Team Lead	Technical Strengths	Professional Strengths	Responsibilities
Chay Hong Hong	**chon0015@student.monash.edu**	**Team Lead**	**Python and Java, Ui design, problem solving**	**Communication, Organisation, Teamwork.**	**Team coordination, system design, integration**
Josh Brian Saputra	**jsap0007@student.monash.edu**	**Developer**	**Coding, problem solving, design**	**Adaptability, teamwork**	**Backend development, core logic implementation**
SimaoHuang	**shua0115@student.monash.edu**	**Developer**	**UML diagrams, system design, coding**	**Teamwork, Collaboration**	**UI design, frontend implementation**

2.2 Team Schedule

During Sprint 2, the DevOops followed a structured schedule to ensure consistent progress and coordination.

The team held regular meetings twice per week, typically at the beginning and middle of the week, to discuss progress, blockers, and upcoming tasks. These meetings were used for sprint planning, task allocation, and quick status updates, Additional communication was conducted through online messaging when needed.

In terms of work schedule, team members worked individually on assigned tasks throughout the week, with integration and testing performed collaboratively towards the end of the week. This allowed each member to focus on their specific responsibilities while ensuring the system components were properly combined.

Workload was distributed based on each member's strengths:

- Backend development and core functionality were handled by Josh
- UI design and frontend implementation were handled by Simao
- System coordination, design documentation, coding support when required and integration were managed by Chay Hong

Task management

2.3 Team Contribution

Figure 2.1 Contribution Analytics

The following figure shows the GitLab Contributor Analytics, illustrating the commit activity of each team member during Sprint 2.



2.4 Link to the Gitlab repository

https://git.infotech.monash.edu/FIT3077/fit3077-s1-2026/assignment-groups/CL_Tuesday11AM_31/project

3 Object-Oriented Design

This section presents the object-oriented design of the Ticket to Ride system using a UML class diagram. The diagram illustrates the main classes, their attributes, methods, relationships, and cardinalities within the system.

The design models the key components of the game, including the central game controller (**Game**), core game entities (**Player, Route, Board**), supporting systems (**TrainDeck, DestinationTicket**), and user interfaces components (**GameScreen, MapView, SetupScreen**). Each class is responsible for a specific aspect of the system, ensuring clear separation of concerns and improving maintainability.

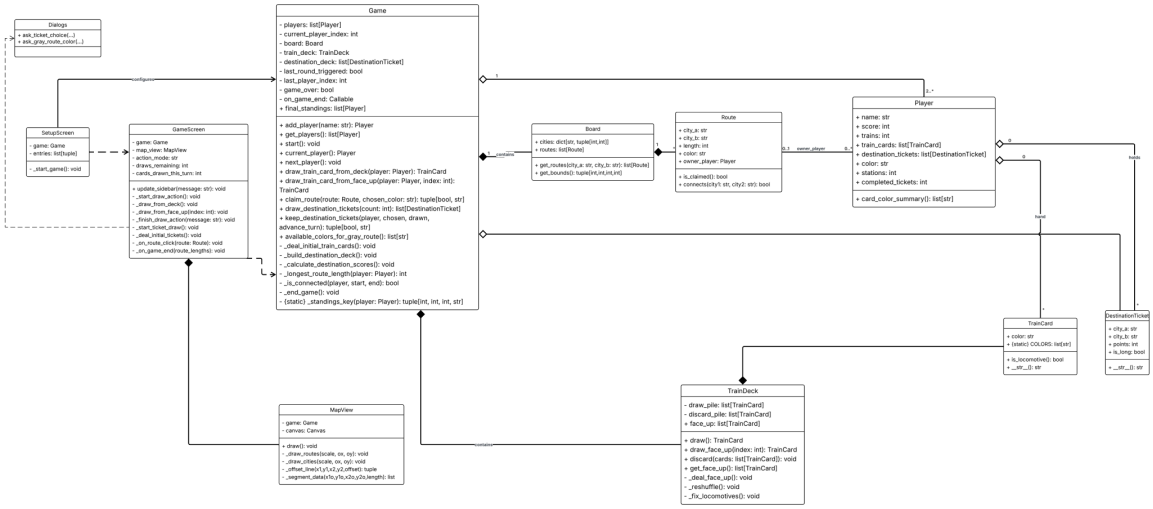


Figure 3.1: UML Class Diagram of the Ticket to Ride System

The diagram shows how the **Game** class manages players, the board, and card decks, acting as the central controller of the system. The **Board** defines cities and routes, while **Route** objects represent connections between cities and can be claimed by players. The **Player** class maintains individual game state, including cards, score, and completed tickets.

Supporting classes such as **TrainDeck** manage card operations, including drawing, discarding, and reshuffling, while **DestinationTicket** represents scoring objectives. The user interface layer is separated into **GameScreen**, **MapView**, and **SetupScreen**, which interact with the core system without embedding game logic.

Relationships between classes include:

- **Game** class contains players, the board and card decks
- **Board** class contains routes
- **Player** class holds train cards and destination tickets
- **Route** is associated with cities and owned by a player
- **UI** classes interact with the **Game** class to perform actions and update the interface
- These relationships and structures reflect the logical interactions within the game and support a modular and extensible system design.

4 Design Rationales

4.1 Overall Design Direction

The system follows an object-oriented design that models the core components of the Ticket to Ride game as separate interacting classes. The design separates responsibilities across game control (Game), game data (Board, Route, Player), and supporting systems (TrainDeck, GameScreen, MapView).

This approach was chosen to ensure:

- Clear separation of concerns
- Easier maintenance and debugging
- Scalability for future features such as UI improvements or rule extensions

The Game class acts as a central controller, coordinating interactions between all major components rather than placing logic inside UI or data classes.

4.2 Key Classes

Game

The Game class is the core controller of the system. It manages:

- Players (players: list[Player])
- Game flow (current_player_index, next_player())
- Game state (game_over, last_round_triggered)
- Core actions such as drawing cards and claiming routes

This class was implemented instead of using standalone methods because it maintains persistent game state across multiple turns. Methods alone cannot store or manage ongoing information such as player turns, decks, and game progress. The Game class also centralises logic, preventing duplication across UI or other classes.

Player

The Player class represents each participant and stores:

- Score, trains, and cards
- Destination tickets
- Completed objectives

This class is necessary because each player has an independent and continuously changing state. Representing players as methods would not allow tracking of individual progress, ownership of routes, or card collections.

TrainDeck

The TrainDeck class manages:

- Draw pile
- Discard pile
- Face-up cards

It includes methods such as draw(), draw_face_up(), discard(), and _reshuffle(). This class was created to encapsulate deck behaviour separately from the Game class. If deck logic were placed inside Game, it would violate the single responsibility principle and make the Game class overly complex. Separating TrainDeck improves modularity and allows easier modification of card mechanics.

DestinationTicket

The DestinationTicket class represents a scoring objective held by a player. It was implemented as a separate class rather than storing ticket data directly on Player because tickets have their own attributes (city_a, city_b, points, is_long) and behaviour (**str**). Keeping them as objects allows them to be shuffled in a deck, drawn, returned, and evaluated independently of the player holding them.

GameScreen, MapView, SetupScreen
The UI layer is split into three classes to separate distinct interface responsibilities. SetupScreen handles player registration before the game starts. GameScreen manages turn flow and player actions during the game. MapView is responsible solely for rendering the board. This separation ensures that no UI class contains game logic, making both the interface and the game rules easier to modify independently.

4.3 Key relationships

1. **Game -> Player (Aggregation, 1 to 2..*)**
   The Game class aggregates Player objects. Players are created externally via add_player() before start() is called, meaning the Game receives them rather than creating them itself. This is aggregation rather than composition because the Game does not own the full lifecycle of a Player. They are registered into the game and managed during gameplay only.
2. **Board -> Route (Composition, 1 to 1..*)**
   The Board class contains Route objects as part of its structure. Routes are defined within the board (routes: list[Route]) and do not exist independently outside of it. This is composition because routes are tightly bound to the board. If the board is removed, its routes no longer exist. A board must always have at least one route to be a valid game board, giving a minimum cardinality of 1.
3. **Player -> TrainCard (Aggregation, 0 to *)**
   The Player class holds a list of TrainCard objects. Cards can exist independently in the deck and are transferred between the deck and the player during the game. This is aggregation rather than composition because cards are not permanently owned by a player. A player can have zero cards at the start of their turn or after spending them to claim a route, so the minimum cardinality is 0.
4. **Player -> DestinationTicket (Aggregation, 0 to *)**
   A player can hold zero or more destination tickets. At the start of the game, tickets are dealt from the deck, so a player begins with zero until the initial deal occurs. This is aggregation because tickets exist in the deck independently before and after being held by a player.
5. **Route -> Player (Association, 0..1 to 0..*)**
   A route is either unclaimed (associated with 0 players) or owned by exactly one player (0..1). A player can own zero or more routes across the board (0..*). This association is not composition or aggregation because route ownership is a temporary game state. The route exists regardless of whether it is claimed.

4.4 Inheritance Decisions

Inheritance was not heavily used in this design.

Although classes such as TrainCard and DestinationTicket share similar characteristics (both represent cards), they were implemented as separate classes rather than using a parent Card class.

This decision was made because:

- Their behaviours are significantly different (route claiming vs scoring objectives)
- Introducing inheritance would add unnecessary abstraction
- The system is clearer when each class has a specific role

Avoiding unnecessary inheritance helps maintain readability and reduces complexity, especially at this stage of development.

4.5 Cardinality Decisions

1. **Game -> Player: 1 to 2..*** The Game must have at least two players to function, as it is a multiplayer game. There is exactly one Game instance managing them.
2. **Player -> TrainCard: 0..*** A player can have zero or more train cards. At the start they may have no cards, and during the game players accumulate and spend them. Using 1..* would be incorrect because it forces a player to always have at least one card, which is not guaranteed.
3. **Board -> Route: 1 to 1..*** A board must always contain at least one route to represent a valid game map. Routes are permanently part of the board structure.
4. **Player -> DestinationTicket: 0..*** Players start with zero tickets until the initial deal. They may accumulate more during the game by drawing from the deck.
5. **Route -> Player: 0..1** A route is either unclaimed (0) or owned by exactly one player (1) at any point in the game.

4.6 Design Patterns

No formal design patterns were implemented in this design. This decision was intentional to keep the system simple and focused on core game functionality during Sprint 2.
However, several patterns were considered:

Singleton Pattern Could be applied to the Game class to ensure only one instance exists. Not used because it reduces flexibility, especially for testing or supporting multiple games.

Observer Pattern Could be used between Game and UI classes (GameScreen, MapView) to update the interface automatically when game state changes. A lightweight equivalent is achieved in the current design through the on_game_end callback attribute on Game, which notifies GameScreen when the game concludes without requiring a full Observer implementation.

Strategy Pattern Could be used for different scoring or rule variations. Not used because the game rules are fixed and do not require interchangeable logic at this stage.

4.7 Summary

The design focuses on separating responsibilities across well-defined classes while keeping the system modular and easy to understand. The **Game** class coordinates the system, while supporting classes such as **Player, Board, Route,** and TrainDeck manage specific aspects of the game.

Relationships and cardinalities were chosen based on how objects interact in the actual game. Inheritance and design patterns were considered but only used where necessary to avoid unnecessary complexity.

5 Supplementary Documentation

The UML class diagram presented in this document represents the final design developed during Sprint 2. Although only one formal diagram is included, it was refined through team discussions and incremental updates throughout the sprint.